MODEL PERFORMANCE WHITEPAPER

YOLO Based Traffic Vehicle Detection,
Tracking and Line-Crossing Analysis

Project: Computer Vision Object Detector & Frame-by-Frame Segmenter Pipeline

Author: Muhammad Ahmed Subhani

Internship: Progree [AI]

Date: 16 Oct 2026

Version: 1.0

Report status: *Measured run metrics included. Accuracy is not claimed without labelled ground truth.*

Table of Contents

Abstract....3
1. Introduction....3
1.1 Objectives....4
1.2 Scope....4
2. System Design and Processing Pipeline....4
2.1 Pipeline stages....4
2.2 Gaussian filtering and adaptive thresholding....5
2.3 Pixel bounds and overlays....5
3. Experimental Setup and Methodology....6
3.1 Test configuration....6
3.2 Frame-level performance measurements....6
3.3 Accuracy evaluation plan....6
4. Results....7
4.1 Runtime performance....7
4.2 Estimated line-crossing counts....7
4.3 Interpretation....8
5. Limitations and Threats to Validity....8
6. Conclusion and Future Work....9
Future-work checklist....9
Appendix A. Reproducibility and Output Files....10
Metric definitions....10
References....10

Abstract

This whitepaper documents a video-based traffic analysis pipeline using the YOLO11n pretrained object detector and OpenCV-based video processing. The system detects vehicle classes, associates detections with track identifiers, estimates line-crossing counts, and records frame-level processing measurements. The evaluated run processed 9,184 frames in 577.106 seconds, with mean processing throughput of 15.914 FPS, mean inference latency of 41.877 ms, and peak process memory usage of 533.84 MB. The pipeline reported 448 line-crossing events: 294 cars, 127 trucks, 27 buses, and zero motorcycles. These are system-generated estimates, not verified ground-truth counts. The report describes the pipeline, filtering stages, measurement method, observed results, limitations, and recommended validation work. Detection accuracy and counting accuracy remain unverified until manually annotated reference data are evaluated.

Keywords: computer vision, YOLO11n, OpenCV, vehicle detection, object tracking, adaptive thresholding, Gaussian blur, line-crossing count, inference latency.

1. Introduction

Traffic video analysis can support traffic-flow monitoring, vehicle counting, and road-use studies. Manual review can be time-consuming, while automated methods can process long recordings consistently. However, automated results depend on camera angle, lighting, occlusion, model confidence, tracking stability, and the definition of a crossing event.

The project requirement was to build a real-time computer vision pipeline using OpenCV and/or a pretrained YOLO model; apply Gaussian filtering and adaptive-threshold processing to video frames; extract shapes using pixel bounds; overlay dynamic count vectors for target classes; and evaluate frame-by-frame performance.

The implemented pipeline uses YOLO11n to detect vehicle classes and a tracking/counting stage to associate detections across frames and count tracks crossing a horizontal line. OpenCV image-processing operations provide a supporting diagnostic view of frame structure. The current run achieved an average processing rate of 15.914 FPS. Whether this is sufficient for a real-time deployment depends on the target frame rate and application latency requirements.

1.1 Objectives

- Detect cars, motorcycles, buses, and trucks in a video stream.
- Track detected objects across consecutive frames.
- Draw bounding boxes, class labels, track IDs, and dynamic crossing totals on the annotated output.
- Apply Gaussian blur and adaptive thresholding as supporting image-processing operations.
- Record frame-level detections and processing time.
- Summarize throughput, latency, CPU utilization, memory usage, and crossing counts.
- Identify limitations and define a reproducible accuracy-validation plan.

1.2 Scope

This report evaluates one video run using the configuration recorded in the output summary. It does not establish generalization to all roads, cameras, weather, lighting, or traffic conditions. Detection and count accuracy remain unverified because a manually labelled ground-truth comparison has not been provided.

2. System Design and Processing Pipeline

2.1 Pipeline stages

- **Video input:** read frames from data/traffic video.mp4.
- **Frame preparation:** decode each frame for processing.
- **Gaussian filtering:** smooth high-frequency image noise before the supporting threshold/shape-analysis stage.
- **Adaptive thresholding:** calculate local thresholds to inspect foreground/background structure under varying illumination.
- **YOLO inference:** run yolo11n.pt with confidence threshold 0.35.
- **Class filtering:** retain car, motorcycle, bus, and truck detections.
- **Tracking:** associate detections across frames and assign track IDs.
- **Line-crossing logic:** count a tracked object when its position crosses the horizontal line at y = 648 pixels.
- **Annotation:** overlay boxes, class labels, track IDs, the counting line, and dynamic class totals.
- **Metrics export:** write predictions, per-frame measurements, and a JSON summary.

2.2 Gaussian filtering and adaptive thresholding

A Gaussian filter smooths local intensity changes and can reduce small image variations before thresholding. Adaptive thresholding calculates a local threshold based on a neighbourhood rather than using one global threshold for the entire image. These operations can help inspect edges, contours, and pixel regions under uneven illumination.

YOLO is the semantic vehicle detector. Gaussian filtering and adaptive thresholding are supporting image-processing operations; they should not be described as the mechanism that produces YOLO's semantic class predictions. If the implementation uses the thresholded image only for a diagnostic mask, state that distinction clearly.

2.3 Pixel bounds and overlays

Each predicted bounding box is represented by pixel coordinates (x1, y1, x2, y2). Box width and height are x2 − x1 and y2 − y1. The centre is ((x1+x2)/2, (y1+y2)/2). The tracker can use the centre and/or overlap between boxes to associate an object with a previous track. The annotation layer uses these coordinates to draw the box and position class and count text.

Implementation details:

Gaussian kernel size: 5 × 5

Adaptive threshold method: Gaussian adaptive thresholding
cv2.ADAPTIVE_THRESH_GAUSSIAN_C

Minimum contour area: N/A

Tracker minimum IoU: 0.05

Tracker maximum centroid distance: 90 pixels

Video resolution: 1920 × 1080 pixels

Input frame rate: 30 FPS

3. Experimental Setup and Methodology

3.1 Test configuration

Parameter	Recorded value
Input video	data/traffic video.mp4
Detector	yolo11n.pt
Confidence threshold	0.35
Frames processed	9,184
Counting line	y = 648 pixels
Vehicle classes	car, motorcycle, bus, truck
Total elapsed time	577.106 seconds
Runtime environment	**CPU:** Intel Core i5-9400F, **GPU:** NVIDIA GeForce GTX 1060 6 GB, **RAM:** 16 GB, **OS:** Windows 11, **Python version:** Python 3.12.0
Video resolution / source FPS	1920 × 1080 pixels 30 FPS

3.2 Frame-level performance measurements

The run summary records mean inference latency, median inference latency, 95th-percentile inference latency, mean processing FPS, mean detections per frame, mean CPU utilization, and peak process memory. Inference latency refers to the model-inference timing recorded by the application; processing FPS reflects the overall pipeline throughput and may include video decoding, tracking, annotation, and output writing.

The first frame may include model initialization or other startup costs. For fair benchmarking, report whether startup time is included and, where possible, provide both end-to-end elapsed time and steady-state frame-processing statistics.

3.3 Accuracy evaluation plan

The current metrics do not establish precision, recall, F1-score, or mean IoU. To measure detection accuracy, manually annotate a representative sample of frames with vehicle classes and bounding boxes, then match predictions to ground truth using a documented IoU threshold (for example, 0.50). Define matching rules, class agreement, duplicate handling, and confidence threshold before evaluation.

For counting accuracy, manually count crossings in a selected segment and compare them with system output. Report absolute error and, where the manual count is non-zero, percentage error. Include

difficult cases such as occlusion, vehicles entering or leaving the image, dense traffic, and vehicles near the counting line.

4. Results

4.1 Runtime performance

Metric	Measured result
Frames processed	9,184
Total elapsed time	577.106 s (about 9 min 37 s)
Mean inference latency	41.877 ms
Median inference latency	36.310 ms
P95 inference latency	63.424 ms
Mean processing throughput	15.914 FPS
Mean detections per frame	10.604
Mean CPU utilization	443.64%
Peak process memory	533.84 MB

The reported throughput is approximately 16 frames per second. This may be suitable for offline traffic analysis or near-real-time use cases with a relaxed frame-rate target, but it does not demonstrate 30 FPS processing. The CPU figure may exceed 100% because some monitoring tools aggregate utilization across logical cores; document the measurement tool and machine specification before comparing this figure with other systems.

4.2 Estimated line-crossing counts

Vehicle class	Estimated crossings
Car	294
Motorcycle	1
Bus	27
Truck	127
Total	448

The total is the sum of the four reported class counts. The system reports 448 track IDs that crossed the line. These values are estimates, not verified unique physical vehicle counts. ID switches can count one physical vehicle more than once; missed detections or tracking interruptions can also lead to undercounting. The zero motorcycle count should be investigated rather than interpreted as proof that no motorcycles crossed.

4.3 Interpretation

The run demonstrates that the system completed a long video and generated frame-by-frame predictions and performance measurements. The average of 10.604 detections per frame indicates a relatively busy scene in the processed recording, although detection density alone is not a measure of accuracy. The latency distribution shows that typical inference time was below the mean, while slower frames increased the average. The current evidence supports a functional pipeline and an initial performance profile; it does not yet support a numerical claim about detection precision or counting reliability.

5. Limitations and Threats to Validity

- **Single-video evaluation:** one recording cannot establish performance across cameras, road layouts, weather, lighting, or traffic density.
- **No verified ground truth:** precision, recall, F1-score, IoU, and counting error remain unmeasured.
- **Tracking errors:** occlusion, missed detections, and ID switches may affect crossing totals.
- **Confidence threshold:** 0.35 trades off missed detections and false positives; its suitability has not been tuned against a labelled validation set.
- **Hardware dependence:** throughput and latency depend on processor, GPU availability, image size, library versions, and video decoding/output overhead.
- **Startup effects:** first-frame initialization can distort latency averages if included without explanation.
- **Thresholding scope:** adaptive thresholding and Gaussian filtering support image analysis, but they do not independently validate semantic vehicle detection.
- **Counting-line placement:** the line at y = 648 is dependent on source dimensions and viewpoint; it may not represent a meaningful traffic boundary in another video.

6. Conclusion and Future Work

The implemented YOLO11n/OpenCV pipeline processed 9,184 frames in 577.106 seconds and reported average throughput of 15.914 FPS. It produced vehicle detections, track IDs, dynamic line-crossing counts, and frame-level performance logs. The recorded count was 448 estimated crossings across cars, buses, and trucks, with no motorcycle crossings reported.

The next priority is to validate detection and counting against manually labelled ground truth. Additional work should measure accuracy on representative frames, inspect motorcycle detections, assess ID-switch frequency, test alternative confidence thresholds, and benchmark on documented hardware. If a 30 FPS target is required, profiling and optimisation should focus on inference resolution, frame skipping, model selection, and output-writing overhead while checking that accuracy does not degrade.

Future-work checklist

☐ Record CPU, GPU, RAM, operating system, Python, OpenCV, PyTorch, and Ultralytics versions.

☐ Record input resolution, source FPS, Gaussian kernel, and adaptive-threshold settings.

☐ Annotate a representative validation set.

☐ Calculate precision, recall, F1-score, and IoU at a documented matching threshold.

☐ Manually verify crossings in a defined video interval.

☐ Report count error and explain any excluded frames.

☐ Benchmark steady-state latency separately from startup.

☐ Test at least one additional video before making general performance claims.

Appendix A. Reproducibility and Output Files

Run command:

python src/main.py --source "data/traffic video.mp4"

- **Annotated video:** outputs/annotated_traffic.mp4
- **Prediction records:** outputs/predictions.csv
- **Frame metrics:** outputs/frame_metrics.csv
- **Summary:** outputs/summary.json
- **Tests:** tests/test_tracker.py
- **Evaluation report:** reports/WHITEPAPER

Metric definitions

- Inference latency: time spent in detector inference as measured by the application.
- Processing FPS: processed frames divided by elapsed processing time, subject to the implementation's timing definition.
- P95 latency: 95th percentile of recorded latency values.
- Precision = TP / (TP + FP).
- Recall = TP / (TP + FN).
- F1-score = harmonic mean of precision and recall.
- IoU = intersection area divided by union area for predicted and ground-truth boxes.
- Counting absolute error = | system count − manual count | .

References

[1] U. U. Y. documentation.. [Online]. Available: https://docs.ultralytics.com/.

[2] O. O. documentation.. [Online]. Available: https://docs.opencv.org/.